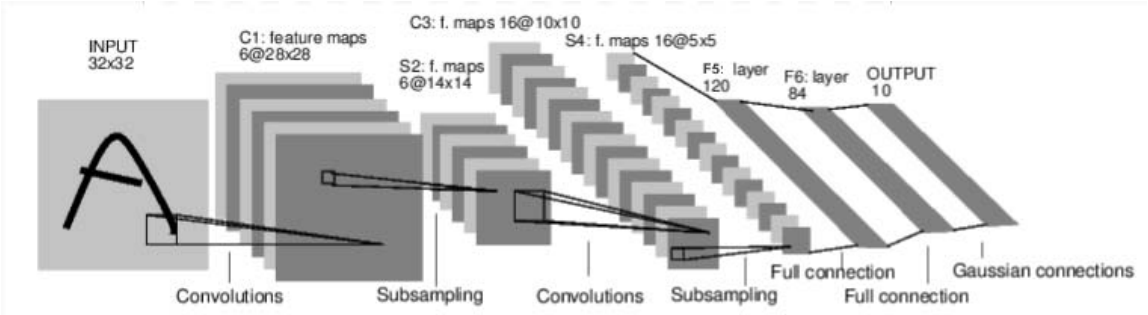
Neural Networks



A typical training procedure for a neural network is as follows:

- Define the neural network that has some learnable parameters (or weights)
- Iterate over a dataset of inputs
- Process input through the network
- Compute the loss (how far is the output from being correct)
- Propagate gradients back into the network's parameters
- Update the weights of the network, typically using a simple update rule:

Define the network

Let's define this network:

```
In [ ]: import torch
        import torch.nn as nn
        import torch.nn.functional as F
        print(torch.__version__)
```

```
2.7.0+cpu
```

We want to be able to train our model on a hardware accelerator like the GPU or MPS, if available.

Let's check to see if `torch.cuda` or `torch.backends.mps` are available, otherwise we use the CPU.

```
In [2]: device = (
            "cuda"
            if torch.cuda.is_available()
            else "mps"
            if torch.backends.mps.is_available()
            else "cpu"
```

```
)
print(f"Using {device} device")
```

```
Using cpu device
```

NN Components

let us examine the component layers of NN

Module

In [3]:
```
nn.Module?
```

```
Init signature: nn.Module(*args, **kwargs) -> None
Docstring:
Base class for all neural network modules.

Your models should also subclass this class.

Modules can also contain other Modules, allowing them to be nested in
a tree structure. You can assign the submodules as regular attributes::

    import torch.nn as nn
    import torch.nn.functional as F

    class Model(nn.Module):
        def __init__(self) -> None:
            super().__init__()
            self.conv1 = nn.Conv2d(1, 20, 5)
            self.conv2 = nn.Conv2d(20, 20, 5)

        def forward(self, x):
            x = F.relu(self.conv1(x))
            return F.relu(self.conv2(x))

Submodules assigned in this way will be registered, and will also have their
parameters converted when you call :meth:`to`, etc.

.. note::
    As per the example above, an ``__init__()`` call to the parent class
    must be made before assignment on the child.

:ivar training: Boolean represents whether this module is in training or
                evaluation mode.
:vartype training: bool
Init docstring: Initialize internal Module state, shared by both nn.Module and Scrip
tModule.
File:           c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\nn\mod
ules\module.py
Type:           type
Subclasses:     Identity, Linear, Bilinear, Threshold, ReLU, RReLU, Hardtanh, Sigmoi
d, Hardsigmoid, Tanh, ...
```

Layers

Linear

In [4]:
```
nn.Linear??
```

```
Init signature:
nn.Linear(
    in_features: int,
    out_features: int,
    bias: bool = True,
    device=None,
    dtype=None,
) -> None
Source:
class Linear(Module):
    r"""Applies an affine linear transformation to the incoming data: :math:`y = xA^
T + b`.

    This module supports :ref:`TensorFloat32<tf32_on_ampere>`.

    On certain ROCm devices, when using float16 inputs this module will use :ref:`di
fferent precision<fp16_on_mi200>` for backward.

    Args:
        in_features: size of each input sample
        out_features: size of each output sample
        bias: If set to ``False``, the layer will not learn an additive bias.
            Default: ``True``

    Shape:
        - Input: :math:`(*, H_\text{in})` where :math:`*` means any number of
          dimensions including none and :math:`H_\text{in} = \text{in\_features}`.
        - Output: :math:`(*, H_\text{out})` where all but the last dimension
          are the same shape as the input and :math:`H_\text{out} = \text{out\_featu
res}`.

    Attributes:
        weight: the learnable weights of the module of shape
            :math:`(\text{out\_features}, \text{in\_features})`. The values are
            initialized from :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})`, where
            :math:`k = \frac{1}{\text{in\_features}}`
        bias:   the learnable bias of the module of shape :math:`(\text{out\_feature
s})`.
                If :attr:`bias` is ``True``, the values are initialized from
                :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})` where
                :math:`k = \frac{1}{\text{in\_features}}`

    Examples::

        >>> m = nn.Linear(20, 30)
        >>> input = torch.randn(128, 20)
        >>> output = m(input)
        >>> print(output.size())
        torch.Size([128, 30])
    """

    __constants__ = ["in_features", "out_features"]
    in_features: int
    out_features: int
    weight: Tensor
```

```
    def __init__(
        self,
        in_features: int,
        out_features: int,
        bias: bool = True,
        device=None,
        dtype=None,
    ) -> None:
        factory_kwargs = {"device": device, "dtype": dtype}
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.weight = Parameter(
            torch.empty((out_features, in_features), **factory_kwargs)
        )
        if bias:
            self.bias = Parameter(torch.empty(out_features, **factory_kwargs))
        else:
            self.register_parameter("bias", None)
        self.reset_parameters()

    def reset_parameters(self) -> None:
        # Setting a=sqrt(5) in kaiming_uniform is the same as initializing with
        # uniform(-1/sqrt(in_features), 1/sqrt(in_features)). For details, see
        # https://github.com/pytorch/pytorch/issues/57109
        init.kaiming_uniform_(self.weight, a=math.sqrt(5))
        if self.bias is not None:
            fan_in, _ = init._calculate_fan_in_and_fan_out(self.weight)
            bound = 1 / math.sqrt(fan_in) if fan_in > 0 else 0
            init.uniform_(self.bias, -bound, bound)

    def forward(self, input: Tensor) -> Tensor:
        return F.linear(input, self.weight, self.bias)

    def extra_repr(self) -> str:
        return f"in_features={self.in_features}, out_features={self.out_features}, b
ias={self.bias is not None}"
File:           c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\nn\mod
ules\linear.py
Type:           type
Subclasses:     NonDynamicallyQuantizableLinear, LazyLinear, Linear, LinearBn1d, Lin
ear
```

`torch.empty()` is a PyTorch function that returns a tensor filled with uninitialized data.

This means that memory is allocated for the tensor, but its values are not set to any specific initial state (like zeros or random numbers).

The content of an empty tensor will be whatever data was **already present in the allocated memory block** before it was assigned to the tensor.

In [5]:
```
torch.empty?
```

```
Docstring:
empty(*size, *, out=None, dtype=None, layout=torch.strided, device=None, requires_gr
ad=False, pin_memory=False, memory_format=torch.contiguous_format) -> Tensor

Returns a tensor filled with uninitialized data. The shape of the tensor is
defined by the variable argument :attr:`size`.

.. note::
    If :func:`torch.use_deterministic_algorithms()` and
    :attr:`torch.utils.deterministic.fill_uninitialized_memory` are both set to
    ``True``, the output tensor is initialized to prevent any possible
    nondeterministic behavior from using the data as an input to an operation.
    Floating point and complex tensors are filled with NaN, and integer tensors
    are filled with the maximum value.

Args:
    size (int...): a sequence of integers defining the shape of the output tensor.
        Can be a variable number of arguments or a collection like a list or tuple.

Keyword args:
    out (Tensor, optional): the output tensor.
    dtype (:class:`torch.dtype`, optional): the desired data type of returned tenso
r.
        Default: if ``None``, uses a global default (see :func:`torch.set_default_dt
ype`).
    layout (:class:`torch.layout`, optional): the desired layout of returned Tensor.
        Default: ``torch.strided``.
    device (:class:`torch.device`, optional): the desired device of returned tensor.
        Default: if ``None``, uses the current device for the default tensor type
        (see :func:`torch.set_default_device`). :attr:`device` will be the CPU
        for CPU tensor types and the current CUDA device for CUDA tensor types.
    requires_grad (bool, optional): If autograd should record operations on the
        returned tensor. Default: ``False``.
    pin_memory (bool, optional): If set, returned tensor would be allocated in
        the pinned memory. Works only for CPU tensors. Default: ``False``.
    memory_format (:class:`torch.memory_format`, optional): the desired memory forma
t of
        returned Tensor. Default: ``torch.contiguous_format``.

Example::

    >>> torch.empty((2,3), dtype=torch.int64)
    tensor([[ 9.4064e+13,  2.8000e+01,  9.3493e+13],
            [ 7.5751e+18,  7.1428e+18,  7.5955e+18]])
Type:      builtin_function_or_method
```

In [6]:
```
print(torch.empty(2,3))
```

```
tensor([[-5.2968e-23,  1.7740e-42,  0.0000e+00],
        [ 0.0000e+00,  0.0000e+00,  0.0000e+00]])
```

In [7]:
```
torch.nn.Parameter?
```

```
Init signature: torch.nn.Parameter(data=None, requires_grad=True)
Docstring:
A kind of Tensor that is to be considered a module parameter.

Parameters are :class:`~torch.Tensor` subclasses, that have a
very special property when used with :class:`Module` s - when they're
assigned as Module attributes they are automatically added to the list of
its parameters, and will appear e.g. in :meth:`~Module.parameters` iterator.
Assigning a Tensor doesn't have such effect. This is because one might
want to cache some temporary state, like last hidden state of the RNN, in
the model. If there was no such class as :class:`Parameter`, these
temporaries would get registered too.

Args:
    data (Tensor): parameter tensor.
    requires_grad (bool, optional): if the parameter requires gradient. Note that
        the torch.no_grad() context does NOT affect the default behavior of
        Parameter creation--the Parameter will still have `requires_grad=True` in
        :class:`~no_grad` mode. See :ref:`locally-disable-grad-doc` for more
        details. Default: `True`
File:           c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\nn\par
ameter.py
Type:           _ParameterMeta
Subclasses:     UninitializedParameter
```

Notice in `nn.Parameter` the default `requires_grad=True`

Example on `Linear`:

In [8]:
```
input = torch.rand(3,2,4) # C x H x W
print("input:")
print(input)

LinearLayer=nn.Linear(in_features=4, out_features=3, bias=True)

print("\n Linear:")
print(LinearLayer)

print("\n LinearLayer weights:")
print(LinearLayer.weight)

print("\n bias:")
print(LinearLayer.bias)

out = LinearLayer(input)
print(out.size())
print("out:")
print(out)
```

```
input:
tensor([[[0.5476, 0.6729, 0.2962, 0.4869],
         [0.8780, 0.6411, 0.8071, 0.1395]],

        [[0.7755, 0.1345, 0.8033, 0.4190],
         [0.4526, 0.8072, 0.0381, 0.1334]],

        [[0.0658, 0.1333, 0.5631, 0.4907],
         [0.2110, 0.5067, 0.6822, 0.6613]]])

 Linear:
Linear(in_features=4, out_features=3, bias=True)

 LinearLayer weights:
Parameter containing:
tensor([[ 0.1320, -0.1575, -0.3313,  0.3007],
        [-0.1473, -0.3256, -0.1885,  0.3220],
        [ 0.4837, -0.0610,  0.2974,  0.4634]], requires_grad=True)

 bias:
Parameter containing:
tensor([ 0.3208, -0.0876, -0.2582], requires_grad=True)
torch.Size([3, 2, 3])
out:
tensor([[[ 0.3354, -0.2863,  0.2793],
         [ 0.1103, -0.5328,  0.4320]],

        [[ 0.2619, -0.2620,  0.5417],
         [ 0.2809, -0.3812, -0.0154]],

        [[ 0.2695, -0.0888,  0.1603],
         [ 0.2417, -0.1992,  0.3222]]], grad_fn=<ViewBackward0>)
```

Convolutional Layer

In [9]:
```
nn.Conv2d?
```

```
Init signature:
nn.Conv2d(
    in_channels: int,
    out_channels: int,
    kernel_size: Union[int, tuple[int, int]],
    stride: Union[int, tuple[int, int]] = 1,
    padding: Union[str, int, tuple[int, int]] = 0,
    dilation: Union[int, tuple[int, int]] = 1,
    groups: int = 1,
    bias: bool = True,
    padding_mode: str = 'zeros',
    device=None,
    dtype=None,
) -> None
Docstring:
Applies a 2D convolution over an input signal composed of several input
planes.

In the simplest case, the output value of the layer with input size
:math:`(N, C_{\text{in}}, H, W)` and output :math:`(N, C_{\text{out}}, H_{\text{ou
t}}, W_{\text{out}})`
can be precisely described as:

.. math::
    \text{out}(N_i, C_{\text{out}_j}) = \text{bias}(C_{\text{out}_j}) +
    \sum_{k = 0}^{C_{\text{in}} - 1} \text{weight}(C_{\text{out}_j}, k) \star \text
{input}(N_i, k)


where :math:`\star` is the valid 2D `cross-correlation`_ operator,
:math:`N` is a batch size, :math:`C` denotes a number of channels,
:math:`H` is a height of input planes in pixels, and :math:`W` is
width in pixels.


This module supports :ref:`TensorFloat32<tf32_on_ampere>`.

On certain ROCm devices, when using float16 inputs this module will use :ref:`differ
ent precision<fp16_on_mi200>` for backward.

* :attr:`stride` controls the stride for the cross-correlation, a single
  number or a tuple.

* :attr:`padding` controls the amount of padding applied to the input. It
  can be either a string {'valid', 'same'} or an int / a tuple of ints giving the
  amount of implicit padding applied on both sides.

* :attr:`dilation` controls the spacing between the kernel points; also
  known as the à trous algorithm. It is harder to describe, but this `link`_
  has a nice visualization of what :attr:`dilation` does.


* :attr:`groups` controls the connections between inputs and outputs.
  :attr:`in_channels` and :attr:`out_channels` must both be divisible by
  :attr:`groups`. For example,
```

```
    * At groups=1, all inputs are convolved to all outputs.
    * At groups=2, the operation becomes equivalent to having two conv
      layers side by side, each seeing half the input channels
      and producing half the output channels, and both subsequently
      concatenated.
    * At groups= :attr:`in_channels`, each input channel is convolved with
      its own set of filters (of size
      :math:`\frac{\text{out\_channels}}{\text{in\_channels}}`).

The parameters :attr:`kernel_size`, :attr:`stride`, :attr:`padding`, :attr:`dilation
` can either be:

    - a single ``int`` -- in which case the same value is used for the height and wi
dth dimension
    - a ``tuple`` of two ints -- in which case, the first `int` is used for the heig
ht dimension,
      and the second `int` for the width dimension

Note:
    When `groups == in_channels` and `out_channels == K * in_channels`,
    where `K` is a positive integer, this operation is also known as a "depthwise co
nvolution".

    In other words, for an input of size :math:`(N, C_{in}, L_{in})`,
    a depthwise convolution with a depthwise multiplier `K` can be performed with th
e arguments
    :math:`(C_\text{in}=C_\text{in}, C_\text{out}=C_\text{in} \times \text{K}, ...,
\text{groups}=C_\text{in})`.

Note:
    In some circumstances when given tensors on a CUDA device and using CuDNN, this
operator may select a nondeterministic algorithm to increase performance. If this is
undesirable, you can try to make the operation deterministic (potentially at a perfo
rmance cost) by setting ``torch.backends.cudnn.deterministic = True``. See :doc:`/no
tes/randomness` for more information.

Note:
    ``padding='valid'`` is the same as no padding. ``padding='same'`` pads
    the input so the output has the shape as the input. However, this mode
    doesn't support any stride values other than 1.

Note:
    This module supports complex data types i.e. ``complex32, complex64, complex128`
`.

Args:
    in_channels (int): Number of channels in the input image
    out_channels (int): Number of channels produced by the convolution
    kernel_size (int or tuple): Size of the convolving kernel
    stride (int or tuple, optional): Stride of the convolution. Default: 1
    padding (int, tuple or str, optional): Padding added to all four sides of
        the input. Default: 0
    dilation (int or tuple, optional): Spacing between kernel elements. Default: 1
    groups (int, optional): Number of blocked connections from input
        channels to output channels. Default: 1
    bias (bool, optional): If ``True``, adds a learnable bias to the
```

```
        output. Default: ``True``
    padding_mode (str, optional): ``'zeros'``, ``'reflect'``,
        ``'replicate'`` or ``'circular'``. Default: ``'zeros'``


Shape:
    - Input: :math:`(N, C_{in}, H_{in}, W_{in})` or :math:`(C_{in}, H_{in}, W_{in})`
    - Output: :math:`(N, C_{out}, H_{out}, W_{out})` or :math:`(C_{out}, H_{out}, W_
{out})`, where

      .. math::
          H_{out} = \left\lfloor\frac{H_{in}  + 2 \times \text{padding}[0] - \text{d
ilation}[0]
                    \times (\text{kernel\_size}[0] - 1) - 1}{\text{stride}[0]} + 1\r
ight\rfloor

      .. math::
          W_{out} = \left\lfloor\frac{W_{in}  + 2 \times \text{padding}[1] - \text{d
ilation}[1]
                    \times (\text{kernel\_size}[1] - 1) - 1}{\text{stride}[1]} + 1\r
ight\rfloor

Attributes:
    weight (Tensor): the learnable weights of the module of shape
        :math:`(\text{out\_channels}, \frac{\text{in\_channels}}{\text{groups}},`
        :math:`\text{kernel\_size[0]}, \text{kernel\_size[1]})`.
        The values of these weights are sampled from
        :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})` where
        :math:`k = \frac{groups}{C_\text{in} * \prod_{i=0}^{1}\text{kernel\_size}
[i]}`
    bias (Tensor):   the learnable bias of the module of shape
        (out_channels). If :attr:`bias` is ``True``,
        then the values of these weights are
        sampled from :math:`\mathcal{U}(-\sqrt{k}, \sqrt{k})` where
        :math:`k = \frac{groups}{C_\text{in} * \prod_{i=0}^{1}\text{kernel\_size}
[i]}`

Examples:

    >>> # With square kernels and equal stride
    >>> m = nn.Conv2d(16, 33, 3, stride=2)
    >>> # non-square kernels and unequal stride and with padding
    >>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2))
    >>> # non-square kernels and unequal stride and with padding and dilation
    >>> m = nn.Conv2d(16, 33, (3, 5), stride=(2, 1), padding=(4, 2), dilation=(3,
1))
    >>> input = torch.randn(20, 16, 50, 100)
    >>> output = m(input)

.. _cross-correlation:
    https://en.wikipedia.org/wiki/Cross-correlation

.. _link:
    https://github.com/vdumoulin/conv_arithmetic/blob/master/README.md
Init docstring: Initialize internal Module state, shared by both nn.Module and Scrip
tModule.
```

```
File:           c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\nn\mod
ules\conv.py
Type:           type
Subclasses:     LazyConv2d, Conv2d, ConvBn2d, Conv2d
```

Example on `Conv2d` :

In [10]:
```
input = torch.rand(1,4,4)
print("input:")
print(input)

convLayer=nn.Conv2d(in_channels=1, out_channels=1, kernel_size = 2, stride=1, bias=

print("\n kernel_size:")
print(convLayer.kernel_size)

print("\n convLayer:")
print(convLayer)

print("\n convLayer weights:")
print(convLayer.weight)

print("\n bias:")
print(convLayer.bias)

twoD_out = convLayer(input)
print(twoD_out.size())
print("twoD_out:")
print(twoD_out)
```

```
input:
tensor([[[0.0192, 0.3332, 0.8673, 0.0309],
         [0.9395, 0.6167, 0.6871, 0.3776],
         [0.8227, 0.6365, 0.8737, 0.0658],
         [0.1807, 0.8593, 0.5544, 0.0250]]])

 kernel_size:
(2, 2)

 convLayer:
Conv2d(1, 1, kernel_size=(2, 2), stride=(1, 1))

 convLayer weights:
Parameter containing:
tensor([[[[ 0.4091,  0.4579],
          [-0.2637,  0.3900]]]], requires_grad=True)

 bias:
Parameter containing:
tensor([0.2650], requires_grad=True)
torch.Size([1, 3, 3])
twoD_out:
tensor([[[0.4182, 0.9038, 0.6000],
         [0.9630, 1.0048, 0.5142],
         [1.1805, 0.9150, 0.5160]]], grad_fn=<SqueezeBackward1>)
```

Pooling

In [11]:
```
F.max_pool2d??
```

```
Signature: F.max_pool2d(*args, **kwargs)
Docstring:
max_pool2d(input, kernel_size, stride=None, padding=0, dilation=1, ceil_mode=False,
return_indices=False)

Applies a 2D max pooling over an input signal composed of several input
planes.

.. note::
    The order of :attr:`ceil_mode` and :attr:`return_indices` is different from
    what seen in :class:`~torch.nn.MaxPool2d`, and will change in a future release.

See :class:`~torch.nn.MaxPool2d` for details.

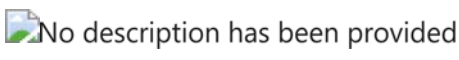
Args:
    input: input tensor :math:`(\text{minibatch} , \text{in\_channels} , iH , iW)`,
minibatch dim optional.
    kernel_size: size of the pooling region. Can be a single number or a
        tuple `(kH, kW)`
    stride: stride of the pooling operation. Can be a single number or a
        tuple `(sH, sW)`. Default: :attr:`kernel_size`
    padding: Implicit negative infinity padding to be added on both sides, must be >
= 0 and <= kernel_size / 2.
    dilation: The stride between elements within a sliding window, must be > 0.
    ceil_mode: If ``True``, will use `ceil` instead of `floor` to compute the output
shape. This
                ensures that every element in the input tensor is covered by a slidin
g window.
    return_indices: If ``True``, will return the argmax along with the max values.
                    Useful for :class:`torch.nn.functional.max_unpool2d` later
Source:
    def fn(*args, **kwargs):
        dispatch_flag = default
        if arg_name in kwargs:
            dispatch_flag = kwargs[arg_name]
        elif arg_index < len(args):
            dispatch_flag = args[arg_index]

        if dispatch_flag:
            return if_true(*args, **kwargs)
        else:
            return if_false(*args, **kwargs)
File:      c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\_jit_intern
al.py
Type:      function
```

Example on `max_pool2d`:

In [12]:
```
x = F.max_pool2d(twoD_out, kernel_size=2, stride=1, padding=0, dilation=1, ceil_mod
print(x)
```

```
tensor([[[1.0048, 1.0048],
         [1.1805, 1.0048]]], grad_fn=<MaxPool2DWithIndicesBackward0>)
```



No description has been provided for this image

In [13]:
```
F.relu??
```

```
Signature: F.relu(input: torch.Tensor, inplace: bool = False) -> torch.Tensor
Source:
def relu(input: Tensor, inplace: bool = False) -> Tensor:  # noqa: D400,D402
    r"""relu(input, inplace=False) -> Tensor

    Applies the rectified linear unit function element-wise. See
    :class:`~torch.nn.ReLU` for more details.
    """
    if has_torch_function_unary(input):
        return handle_torch_function(relu, (input,), input, inplace=inplace)
    if inplace:
        result = torch.relu_(input)
    else:
        result = torch.relu(input)
    return result
File:      c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\nn\function
al.py
Type:      function
```

Example on `relu`:

In [14]:
```
relu = F.relu (twoD_out)
print(relu)
```

```
tensor([[[0.4182, 0.9038, 0.6000],
         [0.9630, 1.0048, 0.5142],
         [1.1805, 0.9150, 0.5160]]], grad_fn=<ReluBackward0>)
```

`tensor.view()` function

returns the a new tensor with the same data as the self tensor but with different shape

In PyTorch, the `tensor.view(*shape)` method is used to **reshape a tensor without copying its underlying data.**

It creates a new tensor object that points to the exact same memory block as the original tensor, making it an extremely fast and memory-efficient operation.

⚙️ Key Mechanics

- **Shared Memory:** Because it shares memory with the original tensor, any modification you make to the reshaped tensor will immediately change the original tensor.
- **Element Count:** The new shape must have the exact same total number of elements as the original tensor (e.g., a (2x3) tensor has (6) elements and can be viewed as (1x6), (3x2), or (6x1)).
- **The `-1` Shortcut:** You can pass `-1` for **one** of the dimensions, and PyTorch will automatically calculate the correct size for that dimension based on the remaining elements.

https://docs.pytorch.org/docs/2.14/generated/torch.Tensor.view.html

In [15]:
```
# ========================================
# 1. BASIC RESHAPING & THE -1 SHORTCUT
# ========================================
print("--- 1. Basic Reshaping ---")
original = torch.arange(6) # Creates [0, 1, 2, 3, 4, 5]
print(f"Original shape: {original.shape}")

# Reshape to a 2x3 matrix
matrix = original.view(2, 3)
print(f"2x3 View:\n{matrix}")

# Flatten back using -1
flattened = matrix.view(-1)
print(f"Flattened view (-1) shape: {flattened.shape}\n")
```

```
--- 1. Basic Reshaping ---
Original shape: torch.Size([6])
2x3 View:
tensor([[0, 1, 2],
        [3, 4, 5]])
Flattened view (-1) shape: torch.Size([6])
```

In [16]:
```
x= torch.rand(2,4)
print(x.size())

#y = x.view(8)
#y = x.view((8,1))
y = x.view((4,2))
print(x)

print(y)
```

```
torch.Size([2, 4])
tensor([[0.7719, 0.5598, 0.0453, 0.2072],
        [0.7362, 0.1309, 0.0113, 0.1454]])
tensor([[0.7719, 0.5598],
        [0.0453, 0.2072],
        [0.7362, 0.1309],
        [0.0113, 0.1454]])
```

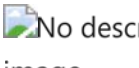
Softmax

The softmax function is a mathematical operation used in machine learning, particularly in the output layer of neural networks, to convert a vector of raw scores (logits) into a probability distribution. It produces a vector of probabilities, where each probability is between 0 and 1, and the sum of all probabilities equals 1.

This function is crucial for multiclass classification tasks, as it provides the model's confidence for each possible outcome.

It is commonly used as an activation function in the output layer of deep learning models for multiclass classification.






```
nn.Softmax?
```

```
Init signature: nn.Softmax(dim: Optional[int] = None) -> None
Docstring:
Applies the Softmax function to an n-dimensional input Tensor.

Rescales them so that the elements of the n-dimensional output Tensor
lie in the range [0,1] and sum to 1.

Softmax is defined as:

.. math::
    \text{Softmax}(x_{i}) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}

When the input Tensor is a sparse tensor then the unspecified
values are treated as ``-inf``.

Shape:
    - Input: :math:`(*)` where `*` means, any number of additional
      dimensions
    - Output: :math:`(*)`, same shape as the input

Returns:
    a Tensor of the same dimension and shape as the input with
    values in the range [0, 1]

Args:
    dim (int): A dimension along which Softmax will be computed (so every slice
        along dim will sum to 1).

.. note::
    This module doesn't work directly with NLLLoss,
    which expects the Log to be computed between the Softmax and itself.
    Use `LogSoftmax` instead (it's faster and has better numerical properties).

Examples::

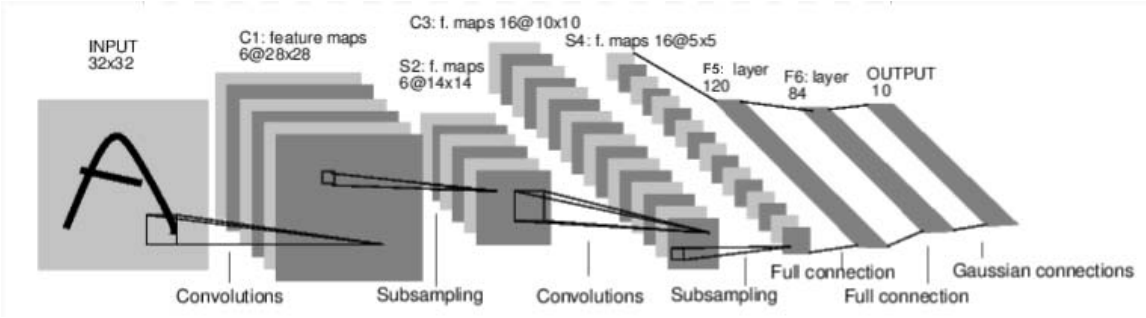
    >>> m = nn.Softmax(dim=1)
    >>> input = torch.randn(2, 3)
    >>> output = m(input)
Init docstring: Initialize internal Module state, shared by both nn.Module and Scrip
tModule.
File:           c:\users\user\anaconda3\envs\env_name\lib\site-packages\torch\nn\mod
ules\activation.py
Type:           type
Subclasses:     Softmax
```

Example on `Softmax`:

In [18]:
```
softmax = nn.Softmax(dim=1)
print(softmax)
outSoftmax = softmax(twoD_out)
print(twoD_out)
print("outSoftmax:")
print(outSoftmax)
```

```
Softmax(dim=1)
tensor([[[0.4182, 0.9038, 0.6000],
         [0.9630, 1.0048, 0.5142],
         [1.1805, 0.9150, 0.5160]]], grad_fn=<SqueezeBackward1>)
outSoftmax:
tensor([[[0.2054, 0.3208, 0.3525],
         [0.3542, 0.3548, 0.3235],
         [0.4403, 0.3244, 0.3241]]], grad_fn=<SoftmaxBackward0>)
```

Build our NN



let us build a network using these types of layers:

In [19]:
```
#import torch
#import torch.nn as nn
#import torch.nn.functional as F

class Net(nn.Module):

    def __init__(self):
        super(Net, self).__init__()
        # 1 input image channel, 6 output channels, 3x3 square convolution
        # kernel
        self.conv1 = nn.Conv2d(1, 6, 3)  # first convolutional layer: Extracts 6 fe
        self.conv2 = nn.Conv2d(6, 16, 3)  # sencond convolutional layer: Takes the
        # an affine operation: y = Wx + b
        self.fc1 = nn.Linear(16 * 6* 6, 120)  # 5*5 from image dimension, 16 for ch
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):

        # Apply ReLU activation to conv1's output, then downsamples spatial dimensi
        x = F.max_pool2d(F.relu(self.conv1(x)), (2, 2)) # Max pooling over a (2, 2)

        # Apply ReLU activation to conv2's output, followed by another 2x2 Max Pool
        x = F.max_pool2d(F.relu(self.conv2(x)), 2) # If the size is a square you ca

        # Flattens the spatial feature maps (16x5x5) into a 1D vector of size 400 p
        x = x.view(-1, self.num_flat_features(x))
        print(x.size())
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
```

```
        x = self.fc3(x)
        return x

    def num_flat_features(self, x):
        size = x.size()[1:]  # all dimensions except the batch dimension
        num_features = 1
        for s in size:
            num_features *= s
        return num_features


net = Net()
print(net)
```

```
Net(
  (conv1): Conv2d(1, 6, kernel_size=(3, 3), stride=(1, 1))
  (conv2): Conv2d(6, 16, kernel_size=(3, 3), stride=(1, 1))
  (fc1): Linear(in_features=576, out_features=120, bias=True)
  (fc2): Linear(in_features=120, out_features=84, bias=True)
  (fc3): Linear(in_features=84, out_features=10, bias=True)
)
```

You just have to define the `forward` function, and the `backward` function (where gradients are computed) is automatically defined for you using `autograd`. You can use any of the Tensor operations in the `forward` function.

The learnable parameters of a model are returned by `net.parameters()`

In [20]:
```
params = list(net.parameters())
#print(params) #printing the weights (elements of all the kernels)
print("parm length:", len(params))
print(params[0].size())  # conv1's .weight
print(params[6].size())  # .weight
```

```
parm length: 10
torch.Size([6, 1, 3, 3])
torch.Size([84, 120])
```

In PyTorch, you can easily access and print the weights of a specific layer in your neural network. Let us print the weights of the convolutional layer named conv2:

Note: The weights before training are got from the empty function (Returns a tensor filled with uninitialized data.)

In [21]:
```
print("\n Conv1 kernel_size:")
print(net.conv1.kernel_size)

print("\n Conv1 bias:")
print(net.conv1.bias)

print("\nConv1 Weights:")
print(net.conv1.weight)
```

```
 Conv1 kernel_size:
(3, 3)

 Conv1 bias:
Parameter containing:
tensor([-0.2667,  0.3227,  0.2820, -0.2850,  0.0365,  0.0655],
       requires_grad=True)

Conv1 Weights:
Parameter containing:
tensor([[[[-0.3021,  0.3011,  0.0606],
          [-0.3283, -0.3322, -0.0959],
          [ 0.1999, -0.3000, -0.0459]]],


        [[[ 0.0846,  0.2740,  0.3087],
          [-0.1184, -0.2178,  0.2435],
          [-0.3298, -0.2293,  0.0069]]],


        [[[ 0.0584, -0.1259,  0.3098],
          [ 0.3055, -0.2119, -0.1344],
          [ 0.3241, -0.1535, -0.0024]]],


        [[[-0.2985, -0.1662,  0.2138],
          [-0.1623, -0.2199,  0.0620],
          [-0.2018, -0.2454,  0.0196]]],


        [[[-0.3175, -0.0524, -0.2005],
          [ 0.0237, -0.2412, -0.1169],
          [-0.3180, -0.0130, -0.1852]]],


        [[[-0.1937, -0.1405,  0.2804],
          [ 0.0990,  0.0735, -0.2470],
          [ 0.2482,  0.2219,  0.0089]]]], requires_grad=True)
```

Let's try a random 32x32 input.

Note: expected input size of this net (Net) is 32x32. To use this net on the MNIST dataset, please resize the images from the dataset to 32x32.

In [22]:
```
input = torch.randn(1, 1, 32, 32) # creat the input as tensor that contains random
print(input)
out = net(input) # pass the
print(out)
```

```
tensor([[[[-1.0534, -0.0950,  0.6443,  ..., -1.1299,  0.6067, -1.0978],
          [-1.6016, -1.0324,  1.2065,  ..., -0.5406,  0.0777,  0.6272],
          [ 0.9852, -0.4373, -0.2041,  ..., -1.8322,  0.2861,  0.5043],
          ...,
          [-0.3888, -0.1753, -0.6269,  ...,  0.0184, -1.2401, -0.7176],
          [-0.3016,  1.1136,  0.9245,  ...,  0.8584,  0.3746,  1.0024],
          [-0.7573,  0.4809,  0.1025,  ..., -1.6104, -0.7185, -0.0531]]]])
torch.Size([1, 576])
tensor([[-0.0091, -0.1056,  0.0579, -0.0392,  0.0177, -0.0005, -0.0596,  0.0013,
          0.1280, -0.1220]], grad_fn=<AddmmBackward0>)
```

The graph of computations that looks like this:

input -> conv2d -> relu -> maxpool2d -> conv2d -> relu -> maxpool2d -> view -> linear -> relu -> linear -> relu -> linear

Zero the gradient buffers of all parameters and backprops with random gradients:

In [23]:
```
net.zero_grad()
out.backward(torch.randn(1, 10))
```

NOTE:

`torch.nn` only supports mini-batches. The entire `torch.nn` package only supports inputs that are a mini-batch of samples, and not a single sample.

For example, `nn.Conv2d` will take in a 4D Tensor of `nSamples x nChannels x Height x Width`.

If you have a single sample, just use `input.unsqueeze(0)` to add a fake batch dimension.

Before proceeding further, let's recap all the classes you've seen so far.

Recap:

- `torch.Tensor` - A *multi-dimensional array* with support for autograd operations like `backward()`. Also *holds the gradient* w.r.t. the tensor.
- `nn.Module` - Neural network module. *Convenient way of encapsulating parameters*, with helpers for moving them to GPU, exporting, loading, etc.
- `nn.Parameter` - A kind of Tensor, that is *automatically registered as a parameter when assigned as an attribute to a* `Module`.
- `autograd.Function` - Implements *forward and backward definitions of an autograd operation*. Every `Tensor` operation creates at least a single `Function` node that connects to functions that created a `Tensor` and *encodes its history*.

At this point, we covered:

- Defining a neural network

- Processing inputs and calling backward

Still Left:

- Computing the loss
- Updating the weights of the network

Loss Function

A loss function takes the (output, target) pair of inputs, and computes a value that estimates how far away the output is from the target.

There are several different [loss functions] (https://pytorch.org/docs/nn.html#loss-functions) under the nn package. A simple loss is: `nn.MSELoss` which computes the mean-squared error between the input and the target.

For example:

In [24]:
```
output = net(input)
print(output)
target = torch.randn(10)  # a dummy target, for example
print(target)
target = target.view(1, -1)  # make it the same shape as output

criterion = nn.MSELoss()

loss = criterion(output, target)
print(loss)
```

```
torch.Size([1, 576])
tensor([[-0.0091, -0.1056,  0.0579, -0.0392,  0.0177, -0.0005, -0.0596,  0.0013,
          0.1280, -0.1220]], grad_fn=<AddmmBackward0>)
tensor([-1.0623, -0.0299,  0.5519,  0.7387, -0.1806,  0.6386, -0.2242,  0.7656,
         0.7414,  0.9587])
tensor(0.4567, grad_fn=<MseLossBackward0>)
```

Now, if you follow `loss` in the backward direction, using its `.grad_fn` attribute, you will see a graph of computations that looks like this:

::

```
input -> conv2d -> relu -> maxpool2d -> conv2d -> relu -> maxpool2d
      -> view -> linear -> relu -> linear -> relu -> linear
      -> MSELoss
      -> loss
```

So, when we call `loss.backward()`, the whole graph is differentiated w.r.t. the loss, and all Tensors in the graph that has `requires_grad=True` will have their `.grad` Tensor accumulated with the gradient.

For illustration, let us follow a few steps backward:

In [25]:
```
print(loss.grad_fn)  # MSELoss
print(loss.grad_fn.next_functions[0][0])  # Linear
print(loss.grad_fn.next_functions[0][0].next_functions[0][0])  # ReLU
```

```
<MseLossBackward0 object at 0x0000021AEF77E8F0>
<AddmmBackward0 object at 0x0000021AEF77E350>
<AccumulateGrad object at 0x0000021AEF77E8F0>
```

Backprop

To backpropagate the error all we have to do is to `loss.backward()`. You need to clear the existing gradients though, else gradients will be accumulated to existing gradients.

Now we shall call `loss.backward()`, and have a look at conv1's bias gradients before and after the backward.

In [26]:
```
print('conv1.bias.grad before zeroing')
print(net.conv1.bias.grad)

print('conv1.weight.grad before zeroing')
print(net.conv1.weight.grad)

net.zero_grad()     # zeroes the gradient buffers of all parameters

print('conv1.bias.grad before backward')
print(net.conv1.bias.grad)

print('conv1.bias.weight before backward')
print(net.conv1.weight.grad)

loss.backward()

print('conv1.bias.grad after backward')
print(net.conv1.bias.grad)


print('conv1.bias.weight after backward')
print(net.conv1.weight.grad)
```

```
conv1.bias.grad before zeroing
tensor([ 0.0597,  0.0096, -0.0856,  0.0212, -0.0929, -0.0276])
conv1.weight.grad before zeroing
tensor([[[[-0.0686,  0.0319,  0.0337],
          [ 0.0275,  0.0072,  0.0403],
          [ 0.0094, -0.0738, -0.0343]]],


        [[[ 0.0651, -0.0238, -0.0255],
          [-0.0232,  0.0432,  0.0381],
          [ 0.0107,  0.0655, -0.0311]]],


        [[[ 0.0137, -0.0024, -0.0241],
          [-0.0205,  0.0685,  0.1866],
          [-0.0787,  0.0014, -0.0205]]],


        [[[ 0.0005,  0.0192,  0.0543],
          [ 0.0252, -0.0111,  0.0192],
          [-0.0710, -0.0385, -0.0273]]],


        [[[ 0.0568, -0.0456,  0.0654],
          [ 0.0103,  0.0462,  0.0216],
          [ 0.0480, -0.0330,  0.0281]]],


        [[[ 0.0098, -0.0852, -0.0205],
          [-0.0536, -0.0483,  0.0382],
          [-0.0825,  0.0605, -0.0175]]]])
conv1.bias.grad before backward
None
conv1.bias.weight before backward
None
conv1.bias.grad after backward
tensor([ 0.0034,  0.0013, -0.0128,  0.0032,  0.0055, -0.0011])
conv1.bias.weight after backward
tensor([[[[-0.0061, -0.0036, -0.0013],
          [ 0.0031,  0.0017, -0.0012],
          [ 0.0043,  0.0011,  0.0043]]],


        [[[-0.0112, -0.0043,  0.0113],
          [ 0.0036, -0.0039, -0.0059],
          [ 0.0086,  0.0075, -0.0042]]],


        [[[ 0.0030,  0.0004, -0.0028],
          [-0.0227,  0.0047, -0.0179],
          [-0.0029,  0.0010,  0.0145]]],


        [[[-0.0013, -0.0199, -0.0103],
          [-0.0024,  0.0047, -0.0009],
          [-0.0080,  0.0039,  0.0049]]],
```

```
[[[ 0.0029, -0.0002, -0.0131],
  [-0.0024,  0.0053,  0.0033],
  [-0.0044, -0.0033,  0.0048]]],


[[[ 0.0074,  0.0060,  0.0043],
  [ 0.0011,  0.0016, -0.0078],
  [ 0.0012, -0.0060,  0.0011]]])
```

Now, we have seen how to use loss functions.

Read Later:

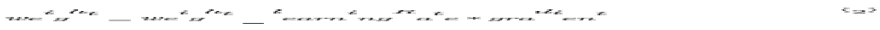
The neural network package contains various modules and loss functions that form the building blocks of deep neural networks. A full list with documentation is [here] (https://pytorch.org/docs/nn).

The only thing left to learn is:

- Updating the weights of the network

Update the weights

The simplest update rule used in practice is the Stochastic Gradient Descent (SGD):



We can implement this using simple Python code:

```
learning_rate = 0.01
for f in net.parameters():
    f.data.sub_(f.grad.data * learning_rate)
```

However, as you use neural networks, you want to use various different update rules such as SGD, Nesterov-SGD, Adam, RMSProp, etc. To enable this, we built a small package: `torch.optim` that implements all these methods. Using it is very simple:

```
import torch.optim as optim

# create your optimizer
optimizer = optim.SGD(net.parameters(), lr=0.01)

# in your training loop:
optimizer.zero_grad()   # zero the gradient buffers
output = net(input)
loss = criterion(output, target)
loss.backward()

# print the bias and weights before update
```

```
print("\n Conv1 bias before update:")
print(net.conv1.bias)

print("\nConv1 Weights before update:")
print(net.conv1.weight.data)

optimizer.step()    # Does the update

# print the bias and weights after update
print("\n Conv1 bias after update:")
print(net.conv1.bias)

print("\nConv1 Weights after update:")
print(net.conv1.weight.data)
```

```
torch.Size([1, 576])

 Conv1 bias before update:
Parameter containing:
tensor([-0.2667,  0.3227,  0.2820, -0.2850,  0.0365,  0.0655],
       requires_grad=True)

Conv1 Weights before update:
tensor([[[[-0.3021,  0.3011,  0.0606],
          [-0.3283, -0.3322, -0.0959],
          [ 0.1999, -0.3000, -0.0459]]],


        [[[ 0.0846,  0.2740,  0.3087],
          [-0.1184, -0.2178,  0.2435],
          [-0.3298, -0.2293,  0.0069]]],


        [[[ 0.0584, -0.1259,  0.3098],
          [ 0.3055, -0.2119, -0.1344],
          [ 0.3241, -0.1535, -0.0024]]],


        [[[-0.2985, -0.1662,  0.2138],
          [-0.1623, -0.2199,  0.0620],
          [-0.2018, -0.2454,  0.0196]]],


        [[[-0.3175, -0.0524, -0.2005],
          [ 0.0237, -0.2412, -0.1169],
          [-0.3180, -0.0130, -0.1852]]],


        [[[-0.1937, -0.1405,  0.2804],
          [ 0.0990,  0.0735, -0.2470],
          [ 0.2482,  0.2219,  0.0089]]]])

 Conv1 bias after update:
Parameter containing:
tensor([-0.2668,  0.3227,  0.2821, -0.2851,  0.0364,  0.0655],
       requires_grad=True)

Conv1 Weights after update:
tensor([[[[-0.3021,  0.3011,  0.0606],
          [-0.3283, -0.3322, -0.0959],
          [ 0.1998, -0.3000, -0.0459]]],


        [[[ 0.0847,  0.2741,  0.3086],
          [-0.1185, -0.2178,  0.2436],
          [-0.3299, -0.2294,  0.0070]]],


        [[[ 0.0583, -0.1259,  0.3098],
          [ 0.3057, -0.2119, -0.1343],
          [ 0.3241, -0.1535, -0.0026]]],
```

```
[[[-0.2985, -0.1660,  0.2139],
  [-0.1623, -0.2200,  0.0620],
  [-0.2017, -0.2455,  0.0196]]],


[[[-0.3176, -0.0523, -0.2003],
  [ 0.0237, -0.2412, -0.1169],
  [-0.3180, -0.0129, -0.1852]]],


[[[-0.1937, -0.1406,  0.2804],
  [ 0.0990,  0.0735, -0.2469],
  [ 0.2481,  0.2219,  0.0089]]]])
```

In [ ]:
```python
import torch.optim as optim

# create your optimizer
optimizer = optim.Adam(net.parameters(), lr=0.01)

# in your training loop:
optimizer.zero_grad()   # zero the gradient buffers
output = net(input)
loss = criterion(output, target)
loss.backward()
optimizer.step()    # Does the update
```

```
torch.Size([1, 576])
```

You have seen how to define neural networks, compute loss and make updates to the weights of the network. One thing is still missing:

Input Data

Generally, when you have to deal with image, text, audio or video data, you can use standard python packages that load data into a numpy array. Then you can convert this array into a `torch.*Tensor`.

- For images, packages such as Pillow, OpenCV are useful
- For audio, packages such as scipy and librosa
- For text, either raw Python or Cython based loading, or NLTK and SpaCy are useful

Let us show our training images, for fun!

In [ ]:
```python
import matplotlib.pyplot as plt
import numpy as np
```

Functions to show an image

In [30]:
```
def imshow(img):
    img = img / 2 + 0.5     # unnormalize
    print("img:")
    print(img)
    npimg = img.numpy()
    print("npimg:")
    print(npimg)
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()
```

In [31]:
```
import torch
import torchvision
print(input)

image = torchvision.utils.make_grid(input)
imshow(image)
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats o
r [0..255] for integers). Got range [-1.206929..2.2129753].
```

```
tensor([[[[-1.0534, -0.0950,  0.6443,  ..., -1.1299,  0.6067, -1.0978],
          [-1.6016, -1.0324,  1.2065,  ..., -0.5406,  0.0777,  0.6272],
          [ 0.9852, -0.4373, -0.2041,  ..., -1.8322,  0.2861,  0.5043],
          ...,
          [-0.3888, -0.1753, -0.6269,  ...,  0.0184, -1.2401, -0.7176],
          [-0.3016,  1.1136,  0.9245,  ...,  0.8584,  0.3746,  1.0024],
          [-0.7573,  0.4809,  0.1025,  ..., -1.6104, -0.7185, -0.0531]]]])
img:
tensor([[[-0.0267,  0.4525,  0.8222,  ..., -0.0649,  0.8033, -0.0489],
         [-0.3008, -0.0162,  1.1032,  ...,  0.2297,  0.5389,  0.8136],
         [ 0.9926,  0.2813,  0.3980,  ..., -0.4161,  0.6431,  0.7521],
         ...,
         [ 0.3056,  0.4123,  0.1865,  ...,  0.5092, -0.1200,  0.1412],
         [ 0.3492,  1.0568,  0.9623,  ...,  0.9292,  0.6873,  1.0012],
         [ 0.1214,  0.7405,  0.5513,  ..., -0.3052,  0.1407,  0.4734]],

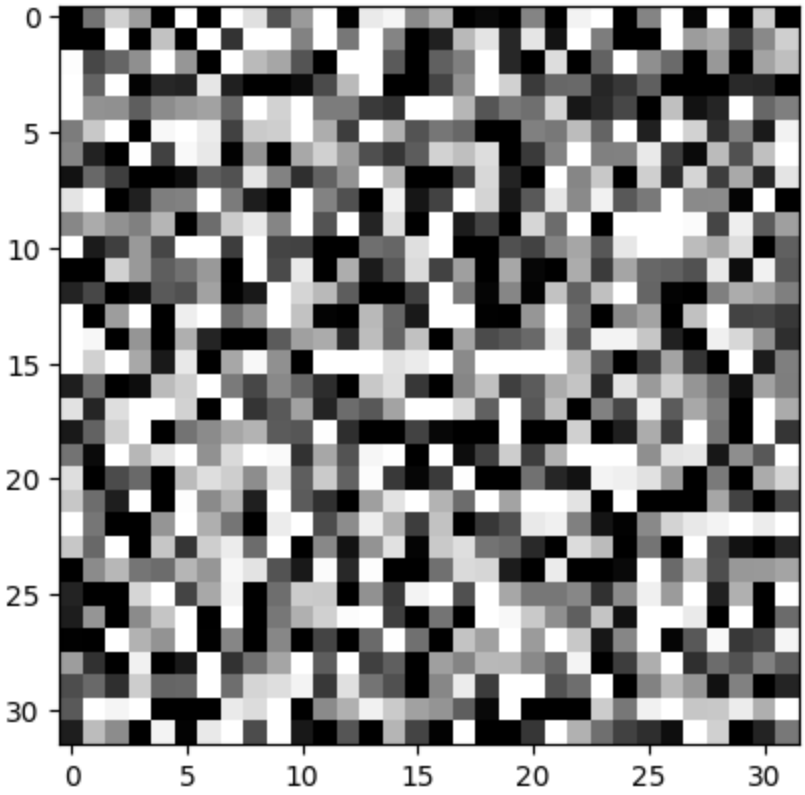
        [[-0.0267,  0.4525,  0.8222,  ..., -0.0649,  0.8033, -0.0489],
         [-0.3008, -0.0162,  1.1032,  ...,  0.2297,  0.5389,  0.8136],
         [ 0.9926,  0.2813,  0.3980,  ..., -0.4161,  0.6431,  0.7521],
         ...,
         [ 0.3056,  0.4123,  0.1865,  ...,  0.5092, -0.1200,  0.1412],
         [ 0.3492,  1.0568,  0.9623,  ...,  0.9292,  0.6873,  1.0012],
         [ 0.1214,  0.7405,  0.5513,  ..., -0.3052,  0.1407,  0.4734]],

        [[-0.0267,  0.4525,  0.8222,  ..., -0.0649,  0.8033, -0.0489],
         [-0.3008, -0.0162,  1.1032,  ...,  0.2297,  0.5389,  0.8136],
         [ 0.9926,  0.2813,  0.3980,  ..., -0.4161,  0.6431,  0.7521],
         ...,
         [ 0.3056,  0.4123,  0.1865,  ...,  0.5092, -0.1200,  0.1412],
         [ 0.3492,  1.0568,  0.9623,  ...,  0.9292,  0.6873,  1.0012],
         [ 0.1214,  0.7405,  0.5513,  ..., -0.3052,  0.1407,  0.4734]]])
npimg:
[[[-0.02667886  0.45248312  0.8221507  ... -0.06493831  0.8033262
   -0.04891521]
  [-0.3008054  -0.0161972   1.1032434  ...  0.22968993  0.53886724
    0.8136176 ]
  [ 0.99257636  0.28133518  0.3979616  ... -0.4160872   0.64306235
    0.75213623]
  ...
  [ 0.30562198  0.41233072  0.18653458 ...  0.5091834  -0.12002754
    0.14122081]
  [ 0.34921068  1.0567888   0.96227384 ...  0.92921287  0.6872779
    1.0012119 ]
  [ 0.12136731  0.7404607   0.551262   ... -0.30518657  0.14074463
    0.47342908]]

 [[-0.02667886  0.45248312  0.8221507  ... -0.06493831  0.8033262
   -0.04891521]
  [-0.3008054  -0.0161972   1.1032434  ...  0.22968993  0.53886724
    0.8136176 ]
  [ 0.99257636  0.28133518  0.3979616  ... -0.4160872   0.64306235
    0.75213623]
  ...
  [ 0.30562198  0.41233072  0.18653458 ...  0.5091834  -0.12002754
    0.14122081]
  [ 0.34921068  1.0567888   0.96227384 ...  0.92921287  0.6872779
```

```
  1.0012119 ]
 [ 0.12136731  0.7404607   0.551262   ... -0.30518657  0.14074463
  0.47342908]]

[[-0.02667886  0.45248312  0.8221507  ... -0.06493831  0.8033262
 -0.04891521]
 [-0.3008054  -0.0161972   1.1032434  ...  0.22968993  0.53886724
  0.8136176 ]
 [ 0.99257636  0.28133518  0.3979616  ... -0.4160872   0.64306235
  0.75213623]
 ...
 [ 0.30562198  0.41233072  0.18653458 ...  0.5091834  -0.12002754
  0.14122081]
 [ 0.34921068  1.0567888   0.96227384 ...  0.92921287  0.6872779
  1.0012119 ]
 [ 0.12136731  0.7404607   0.551262   ... -0.30518657  0.14074463
  0.47342908]]]
```



In [ ]: